PreM-FedIoV: A Novel Federated Reinforcement Learning Framework for Predictive Maintenance in IoV

Lu Yang, Songtao Guo, *Senior Member, IEEE*, Chen-Khong Tham, *Senior Member, IEEE*, Mingyan Li, Guiyan Liu, and Pengzhan Zhou, *Member, IEEE*

***Abstract*—The Internet of Vehicles (IoV) enhances data availability by equipping a plethora of sensors, driving the automotive industry towards data-driven Predictive Maintenance (PreM) models. However, traditional centralized PreM solutions, requiring complete access to training data, raise concerns about data privacy. PreM in the automotive domain is more challenging than in many other fields, partly due to the varying distribution nature of data samples and the limited network connectivity time caused by vehicle mobility. To address these challenges, we propose the PreM-FedIoV framework, extending single-agent Double Deep Q-Network (DDQN) to Multi-Agent Double Deep Q-Network (MADDQN). In each round, each vehicle client uploads a data packet to the server based on the current contention window, containing its local model, local test Mean Absolute Error (MAE), and a timestamp. The server initially performs federated aggregation on the received local models. The MADDQN module then dynamically adjusts the contention window of each vehicle for the next round based on the local test MAE and communication statistical state, aiming to optimize communication costs and predictive performance. Additionally, we utilize NS-3 to create IoV simulations and deploy the PreM-FedIoV framework within NS3-gym. We choose Federated Averaging (FedAvg) and FedAdam following the IEEE 802.11p standard as baselines. The experiments demonstrate significant improvements in our framework compared to state-of-the-art algorithms. On the C-MAPSS dataset, we achieve reductions of up to 10.2% in MAE, 26.31% in average communication clock time per round, and 65.6% in the number of participating clients per round. For the Random Battery Usage dataset, with up to 4.55%, 24.44%, and 36.58% improvements in the respective metrics.**

***Index Terms*—Contention window, federated learning, Internet of Things, Internet of Vehicles, NS-3, NS3-gym, predictive maintenance.**

Manuscript received 3 December 2023; revised 27 April 2024; accepted 17 May 2024. Date of publication 22 May 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272069, Grant 62202073, Grant 62202072, in part by the Natural Science Foundation of Chongqing under Grant CSTB2022NSCQ-MSX1029, and in part by Special Research Funding for Chongqing Postdoctoral Researchers under Grant 2021XM2015. Recommended for acceptance by M. Segata. *(Corresponding authors: Songtao Guo; Mingyan Li.)*

Lu Yang, Songtao Guo, Mingyan Li, Guiyan Liu, and Pengzhan Zhou are with the Key Laboratory of Dependable Service Computing in Cyber-Physical-Society (Ministry of Education), and College of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: guosongtao@cqu.edu.cn; limy2021@cqu.edu.cn).

Chen-Khong Tham is with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 119077.

Digital Object Identifier 10.1109/TMC.2024.3404249

I. Introduction

CONNECTED fleets of vehicles have been extensively integrated into the Industrial Internet of Things (IoT) field globally [1], [2]. Through efficient collaboration, they achieve seamless data collection and extraction, thereby injecting new momentum into the optimization of operational models and the simplification of complex production processes [3]. In this context, vehicle malfunctions pose not only a significant threat to the safety of vehicle owners but also result in substantial labor costs for enterprises. Due to the exponential increase in vehicle-generated data and the significant resurgence of Artificial Intelligence (AI), data-driven Predictive Maintenance (PreM) has emerged as the most efficient solution to address the aforementioned issues [4]. Based on features extracted from state monitoring data, data-driven PreM methods can forecast the Remaining Useful Life (RUL) of components, estimating the time remaining before operational failure and guiding maintenance strategies [5]. The existing methods are primarily implemented by uploading data captured by on-board sensors to a central server for model training. However, transmitting these data wirelessly to the central server incurs significant wireless resource costs and communication delays, especially in the presence of a large number of moving vehicles. On the other hand, data captured by onboard sensors often includes sensitive privacy information, such as vehicle location, driver behavior, and onboard camera data. In recent years, there has been increasing interest in applying Federated Learning (FL) to the IoV [6], [7].

Through the FL framework, all vehicles can train their individual local RUL prediction models based on their local sensor data. Only these local models are uploaded to the server for global model aggregation, instead of transmitting raw training data. The server aggregates these models to form a global model and broadcasts it to all participating vehicles through the Road Side Unit (RSU). This approach simultaneously protects data privacy and reduces communication costs [8], [9]. However, the deployment of FL in the real-world IoV still encounters numerous challenges. First, due to the fact the training data presented by individual clients is collected by the vehicles themselves based on their local environment and usage patterns, the size and distribution of local datasets often vary significantly among different clients. As we continuously fit the model to the heterogeneous

1536-1233 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

local data on different clients, the inconsistency in local objective functions among different clients will ultimately lead to a decline in FL performance [10], [11]. This discrepancy necessitates more rounds of communication before training convergence, and the significant communication overhead is largely attributed to frequent gradient transmission from vehicle clients to the server. The existing research has explored several approaches to addressing the issue of statistical heterogeneity in the FL setting. For example, Li et al. improved the overall convergence stability by incorporating a proximal term into the optimization objective [12]. Yang et al. addressed the Non-Independent and Identically Distributed (non-IID) problem by enabling local collaboration among edge devices [13].

Second, due to the potential large number of participating vehicle clients in FL, when multiple vehicles compete to upload local models based on the IEEE 802.11p protocol, vehicles need to compete for channel resources based on Carrier Sense Multiple Access/Collision Avoidance (CSMA/CA). Vehicles use the same initial contention window size to ensure fair wireless access. However, due to the statistical heterogeneity of local datasets, there is a variation in the bias degree between clients' local models. However the channel access probability is the same for all clients, which potentially leads to a slowdown in convergence speed and an increase in communication costs. Moreover, the CSMA/CA mechanism uses the Binary Exponential Backoff (BEB) algorithm to effectively minimize channel competition conflicts. This results in an increase in wireless access clock time and a waste of wireless resources. Vehicles moving in areas with poor network coverage or between countries experience decreased communication duration, exacerbated by higher speeds. The rise in wireless access clock time limits the iterations within a given connection time, potentially causing delayed model training. This could lead to missed detection of potential faults until after they occur, increasing both maintenance costs and downtime.

Although some FL frameworks have been studied to address statistical heterogeneity [12], [14], most research focused on improving the convergence speed of client scheduling by reducing the number of communication rounds to achieve a certain level of test accuracy. However, this does not reduce the wall clock time consumption during the training process. Deploying these FL frameworks directly into IoV would not only result in a loss of accuracy in the global model and significant communication overhead but also increase vehicle downtime, impacting the normal operation of the fleet as mentioned earlier. On the other hand, current FL algorithms are mainly concentrated on training different datasets and algorithmic improvements, often overlooking the complexities of network behaviors. In the real-world setting, large-scale deployment and validation of these algorithms are often impractical. Therefore, selecting an appropriate network simulator for vehicular network simulation and deploying FL frameworks within it is crucial for the practical implementation of FL.

The reasonable optimization of the contention window is the key to addressing the above challenges. A larger contention window increases the probability of successful local model uploads but also increases wireless access delay, while a smaller contention window increases collision probability but reduces wireless access delay. Utilizing Reinforcement Learning (RL) dynamically allocates different sizes of contention windows to vehicle clients with different degrees of bias in each round to achieve joint optimization of global model accuracy, number of participating clients, and communication clock time. Although some studies have used RL to optimize contention windows and improve system throughput [15], [16], there are fewer studies to be done in the context of the FL framework. In this paper, we propose a FL framework based on Multi-Agent Reinforcement Learning (MARL) deployed in the IoV, which is called the Federated Predictive Maintenance framework for IoV (PreM-FedIoV). This framework efficiently determines the optimal contention window size during runtime, aiming to optimize predictive performance while reducing communication overhead.

Specifically, we extend the single-agent Deep Q-Network (DDQN) to Multi-Agent Deep Q-Network (MADDQN). In each round, MADDQN adaptively determines the optimal contention window size for each vehicle client based on the bias level of its local model and communication statistical state. Vehicle clients keep transmitting data based on the optimal contention window size in each round. To closely simulate the performance of the FL framework used for PreM in the real IoV scenario, we construct a vehicular network based on the IEEE 802.11p communication protocol using existing network components in NS-3. PreM-FedIoV is deployed in the simulated IoV network using the NS3-gym component to validate its performance. To the best of our knowledge, this is the first FL work to enhance performance by adjusting the contention window. Therefore, we use the Federated Averaging (FedAvg) [17] and FedAdam [18] algorithms based on the CSMA/CA mechanism as the baseline, among which the contention window adjustment method is currently the state of the art in IoV [19]. Additionally, we employ Convolutional Neural Network (CNN) and Long Short-Term Memory Network (LSTM) as local models to explore the effectiveness of these two network architectures within the FL framework in the IoV, given their widespread use in the field of PreM [20]. The experiments show that the PreM-FedIoV framework achieves a lower Mean Absolute Error (MAE). Furthermore, fewer vehicle clients are required to participate in each communication round, leading to faster convergence.

The key contributions of our work can be summarized as follows

1) We propose the PreM-FedIoV framework, which is a FL-based RUL prediction system. Within this framework, we assess the performance of both Convolutional Neural Network (CNN) and Long Short-Term Memory Network (LSTM).
2) We extend the single-agent DDQN to MADDQN, where the network dynamically determines the optimal contention window size for each vehicle client in each round. It adapts the size based on the local model's bias level and communication statistics status.
3) We use existing network components in NS-3 to build the IoV network model based on IEEE 802.11p. Furthermore, we deploy the PreM-FedIoV framework in the IoV network through NS3-gym.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

4) We conduct comprehensive experiments on the C-MAPSS dataset and the battery random usage dataset generously provided by NASA. We select FedAvg and FedAdam based on IEEE802.11p as baselines and perform a comprehensive evaluation and analysis based on methods such as communication overhead and predictive accuracy.

The structure of this paper is as follows. Section provides an introduction to the PreM based on FL framework in IoV. Subsequently, Section II provides an overview of related work. Section VII introduces our deep learning approach for predicting RUL. Section III introduces the general framework for deploying FL in IoV and discusses the challenges involved. Section IV introduces the proposed PreM-FedIoV framework and discusses the functionality of server-side dynamic contention window adjustment. Section V also explains how the FL framework is deployed in IoV using NS-3 and NS3-gym. Additionally, the C-MAPSS dataset, as well as data preprocessing and data partitioning method, are described in Section VI. Section VIII presents the experimental setup, results and discussions. Finally, Section IX concludes the paper by summarizing the findings and highlighting future research directions.

II. Related Work

In this section, we provide a brief overview of relevant works closely related to our research.

A. Predictive Maintenance

Data-driven methods like Machine Learning (ML) are increasingly common in industrial PreM. Its core function involves predicting the RUL of equipment based on monitoring data. Subsequently, by utilizing these predictive insights and available maintenance resources, rational maintenance strategies are devised to reduce warranty costs. Data-driven fault diagnosis and prediction research have yielded notable results. For instance, Davide et al. proposed a novel approach combining Long Short-Term Memory (LSTM) neural networks and Bayesian inference for PreM in industrial factories [21]. Bita et al. proposed a classification algorithm called Balanced K-Star (BKS) to address the issue of data imbalance in predictive maintenance applications [22]. Maren et al. proposed a novel deep learning model based on autoencoders and bidirectional gated recurrent unit networks for handling extremely rare fault predictions in aircraft predictive maintenance modeling [23]. Ali et al. implemented PreM in manufacturing systems using an attention-based long short-term memory fully convolutional neural network with adaptive boosting (ALSTM-FCN with AdaBoost) [24]. Wang et al. proposed a DL ensemble model to effectively predict the RUL of aircraft engines. This model comprises a one-dimensional CNN and a Bidirectional Long Short-Term Memory network (Bi-LSTM) with the attention mechanism [25].

To better capture short-term and long-term dependencies in time series data, some studies utilize Transformer encoders for prediction, like [26], [27], [28].

However, these works are all based on centralized learning. From the perspective of protecting user privacy, research on distributed PreM methods is still limited.

B. Federated Learning in IoV

By introducing the FL framework into the IoV, datasets collected from multiple vehicles can be utilized to train ML models, ensuring the privacy of vehicle users and enabling various vehicle applications. For example, Rishu et al. applied FL to driver behavior analysis, classifying driver behavior (safe, unsafe, or fatigued) without sharing raw data [29]. Lian et al. proposed applying FL and model sparsity to optimize Traffic Sign Recognition (TSR) in autonomous vehicles [30]. Kai et al. proposed a FL model based on connection records for classifying vehicular network terminals [31]. Zeng et al. proposed a federated learning algorithm for data privacy protection based on homomorphic encryption, classifying vehicles as emergency or non-emergency [32]. Despite these efforts, FL in the IoV faces challenges such as non-IID, imbalanced data, energy efficiency, and communication bottlenecks. Some existing works have addressed part of these issues. For example, Hu et al. proposed a TOP-K-based vehicle node selection scheme to improve the accuracy of the training model and ensure data reliability [33]. Zhang et al. proposed an energy-efficient FL scheme based on two-stage game theory to reduce the energy consumption associated with frequent model aggregation [34]. Saniya et al. proposed a FL model based on gated recurrent units for resource allocation in the moving small cells network supporting vehicular edge computing [35].

However, the aforementioned efforts often focus on the development of architecture and algorithms. Yet, the actual performance of FL depends on the interaction among data, algorithms, and the network. Currently, research on FL in the IoV has not yet taken into account all these factors.

C. Contention Window Optimization

The contention window is a highly critical factor regulating medium access for network nodes, thus making it a recurring theme in multiple research analyses. For example, Kwon et al. proposed a Q-learning-based contention window adjustment method for wireless body area networks, which adaptively adjusts the contention window based on acknowledged receptions to reduce the number of collisions [36]. Ke et al. used Multi-Agent Deep Reinforcement Learning (MADRL) to optimize the contention window and improve system throughput [37]. Jin et al. proposed a dynamic adjustment method for the minimum contention window based on the number of nodes, aiming to achieve fair channel resource allocation [38]. Subash et al. utilize the DQN to adaptively adjust the contention window based on reward functions, thereby efficiently improving communication efficiency in Flying Ad-Hoc Networks (FANET) [39]. Ke et al. proposed the Smart Exponential-Threshold-Linear backoff algorithm with DQN (SETL-DQN) algorithm to optimize the optimal parameters for contention window thresholds under various network configurations, aiming to increase throughput and reduce collision rates [40]. However, all these methods assume nodes collaborate by either choosing the same minimum contention window or adhering to predetermined behaviors, and none of the works consider the context of FL.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

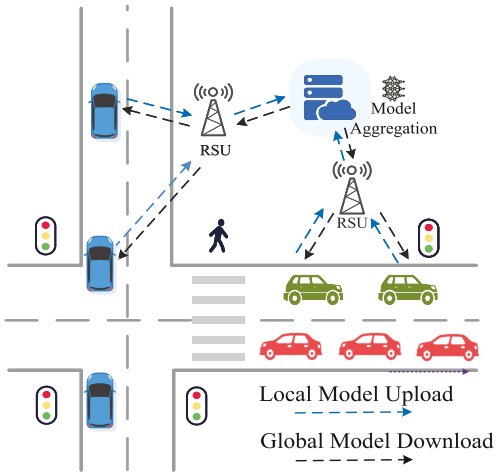


Fig. 1. Federated learning in IoV.

To address the limitations of previous studies, in this work, we deploy the FL framework in IoV network simulation with the goal of PreM, aiming to validate the impact of network status on FL performance. Additionally, we dynamically allocate the optimal contention window size for each vehicle node in the current round using MARL to reduce communication overhead.

III. System Model

Deploying FL within the IoV poses a significant challenge due to its distinctive and complex characteristics. In this section, we will provide a brief overview of the general architecture of FL in IoV. Following that, we will demonstrate how IoV presents challenges to the existing FL framework.

A. Federated Learning in IoV

Today's smart vehicles are equipped with various types of sensors to monitor component status. Additionally, they are outfitted with well-equipped On-Board Units (OBUs), which typically include processors, memory, communication modules, and other components. OBUs can process and cache onboard sensor data, utilizing wireless communication modules to maintain the vehicle's connection with infrastructure [41], [42]. In our specific scenario, a remote cloud server takes on the central server role, while a Road Side Unit (RSU) acts as the relay station responsible for distributing the global model and uploading local models. Vehicles are equipped with OBUs and multiple sensors that operate as clients. These sensors collect component status information and transmit it to the OBU, which then independently trains a local model based on the local sensor data. Moreover, when a vehicle switches to a new RSU coverage area due to movement, it can eliminate the additional traffic setup delay brought by the switch by actively setting flow rules [43].

In the FL framework, each vehicle only needs to transmit its updates to the shared global model to the central server. By employing distributed training at the client level, the central server can achieve highly accurate predictive models while maintaining user data privacy. This setup is illustrated in Fig. 1, featuring a star-shaped network topology comprising K vehicles and a single RSU, which is connected to the central server.

The local dataset of vehicle k can be represented as D_k, and the local objective of each vehicle is to minimize the local empirical risk under its local data distribution. Generally speaking, our aim is to address a distributed optimization problem

$$\min_{\boldsymbol{w}} \frac{1}{K} \sum_{k=1}^{K} E_{(x,y)\sim D_k} \left[L(f_{\boldsymbol{w}}(x), y) \right]. \tag{1}$$

Here, $\boldsymbol{w}$ represents the model parameters, K is the number of clients, D_k denotes the data distribution of the kth client, x and y are input features and the corresponding label, respectively, $f_{\boldsymbol{w}}(x)$ is the result predicted by the model for the input x, and $L(\cdot)$ represents the loss function.

By collectively minimizing this empirical risk, we can achieve a globally optimal or near-optimal model while respecting the privacy constraints of individual clients. Specifically, the steps of FL in IoV are as follows,

- *Step 1:* In each round, the RSU downloads the current global model from the central server and subsequently broadcasts it to the participating vehicles in the network.
- *Step 2:* Each vehicle trains its local model based on its local data for model updates.
- *Step 3:* Each vehicle transmits its updated local model to the central server through RSU.
- *Step 4:* The central server, based on the received local models, excludes stragglers and performs federated averaging to update the global model.

The communication process of FL in IoV can be specifically divided into two phases: communication between the server and RSU, and communication between RSU and vehicle clients. Due to the high-speed wired link used for communication between the server and RSU, the communication overhead of the server-RSU can be negligible [44]. The communication process between vehicles and RSU uses the IEEE 802.11p protocol. Specifically, the basic Media Access Control (MAC) channel access method of IEEE 802.11p is CSMA/CA. The physical layer of IEEE 802.11p is based on Orthogonal Frequency Division Multiplexing (OFDM), and when the MAC layer chooses to send a data frame, it passes the data frame to the physical layer. The physical layer uses OFDM technology to multiplex multiple subcarriers into a single channel to achieve a higher throughput [45].

In this setup, during each round, the RSU initially broadcasts the global model to all client vehicles. When multiple vehicles complete local model training and simultaneously upload their local updates to the RSU according to the current contention window, the Binary Exponential Backoff (BEB) mechanism is applied within the contention window if collisions occur. Once vehicles successfully compete and pass the backoff stage, they can transmit data packets through the medium.

B. Challenges

The performance of FL often depends on the interaction between the actual network environment and the algorithms.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

In vehicular networks, challenges such as the mobility of vehicles, imbalanced and non-IID client data, as well as the BEB mechanism, pose various challenges to the performance of FL, as outlined below.

1) Unbalanced and non-IID Data: Varying data sizes and distributions among clients impact FL due to data collected locally by vehicles based on their unique environments and usage patterns. Inconsistent local objectives across clients result in diminished FL performance [10], requiring additional communication rounds for convergence.

2) FL With Random Access: In CSMA/CA, vehicles use the same initial contention window size for fairness. However, during the local model upload process, conflicts cannot be entirely avoided, and any subset of local models may fail to upload. This can result in slower convergence speeds and higher communication costs. Additionally, while doubling the contention window can prevent conflicts, it prolongs waiting times, wastes channel resources, and increases access latency.

3) High Mobility and Communication Costs: Nodes' high mobility in IoV reduces the available communication time but lowers the number of iterations due to increased access channel delays, resulting in reduced global model accuracy. Additionally, the communication cost during local model upload is proportional to the number of vehicle clients successfully uploading their local models. While the BEB algorithm effectively reduces conflicts, it also involves the participation of a larger number of clients.

IV. PreM-FedIoV Framework Design

PreM utilizes the historical data and operational parameters of vehicles to forecast the occurrence of vehicle failures, assisting vehicle owners in making maintenance decisions promptly and enhancing travel safety and service experience. However, the data generated by vehicles contains a large amount of user privacy information, such as frequently visited locations, daily routines, home and work addresses, and travel routes. In traditional centralized learning approaches, it's common practice to send raw data to the central server, which undoubtedly poses risks of user information leakage. In the context of vehicle PreM, we introduce the FL framework to safeguard user privacy, where users need only upload updates instead of raw data.

Furthermore, to address the challenges outlined in Section III, we formulate the contention window settings for vehicle nodes in the context of FL in IoV as a MADRL problem. Building upon this, we extend the DDQN to a cooperative multi-agent setting and propose the PreM-FedIoV. In this framework, we design a MADDQN scheme to dynamically adjust the contention window size for each vehicle client in each iteration based on the bias level of its local model, effectively allocating distinct wireless access priorities to various vehicle nodes.

In this section, we will first state the formulation of the FL optimization problem in IoV, then introduce our key insights, and finally present the design of the PreM-FedIoV system.

A. Problem Formulation

In this work, we focus on optimizing wireless access during the uplink communication process and assume that all vehicle clients successfully receive the global model during the downlink communication. This assumption is reasonable because the uplink is usually the communication bottleneck due to the limited resources of vehicle clients (e.g., transmission power and bandwidth) [46], [47]. In each round $t \in T$, if client k successfully uploads its local model, the communication delay T_k^t of client k can be expressed as follows

$$T_k^t = T_{Receive}^{k,t} - T_{Send}^{k,t}, \tag{2}$$

where $T_{Send}^{k,t}$ is the local data packet sending time of client k in round t, and $T_{Receive}^{k,t}$ is the local data packet receiving time. If client k's data packet is lost due to collision, $T_k^t = 0$. So the communication delay T^t of each round t can be expressed as

$$T^t = \max\left(T_1^t, T_2^t, \ldots, T_K^t\right). \tag{3}$$

Furthermore, in round t, the number of participating vehicle clients C^t in model aggregation can be represented as

$$C^t = \sum_{k=1}^{K} P_k^t, \tag{4}$$

where binary variable $P_k^t \in \{0, 1\}$ represents whether client k has successfully uploaded the local model in round t. If it is successful, $P_k^t = 1$; otherwise, if a collision occurs and the upload fails, this process incurs no communication cost, and $P_k^t = 0$.

To evaluate the global model's performance, we employ the Mean Absolute Error (MAE) as the evaluation metric, where $MAE(T)$ represents the testing error of the global model on the global test dataset after the last training round. Our FL system optimization problem can be defined as

$$\min_{CW} E\left[w_1 \mathrm{MAE}(T) + w_2 \sum_{t \in T} T^t + w_3 \sum_{t \in T} C^t\right], \tag{5}$$

where matrix $CW = [CW_k^t] \in K \times T$ represents the contention window size for each vehicle client in each round, while w_1, w_2, and w_3 denote the importance of objectives controlled by the FL application designer. Our goal is to minimize the MAE of the global model on the test dataset while simultaneously minimizing the communication wall clock time and communication costs. The expectation is taken over the stochasticity of deep learning training. In the given scenario, since there are no explicit target labels, we employ MARL, allowing the agent to learn based on reward signals provided by the environment, autonomously exploring to find the optimal model.

B. Key Insights

Using RL to optimize the contention window is a well-established research topic [37], [40], [48], [49], [50], [51]. However, to the best of our knowledge, there is currently no prior research on contention window optimization to enhance FL performance within the context of vehicular networks. In the previous section, we examined the impact of non-IID data and radio resource conflicts on FL performance. In fact, a promising approach to mitigate the differences between local objective functions is eliminating biased model updates, which introduces

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

outliers that negatively affect the overall convergence speed. Given this, our primary insights are as follows:

- Within the FL framework, particularly when facing the challenge of non-IID data distributions among clients, controlled collisions can help to eliminate outliers, thereby accelerating convergence and reducing communication costs. This is because larger contention windows decrease the probability of collisions by reducing the likelihood of clients choosing the same random backoff time during channel access competition.
  We can allocate larger contention windows to clients with less biased local models, and smaller contention windows to clients with more biased local models. Consequently, local models with greater bias become less likely to participate in global aggregation due to the increased collision probability. This process, to some extent, parallels the client selection procedure in [52]. Employing MARL for adaptive adjustment of contention window sizes, instead of simple doubling operations, also aids in optimizing the communication wall clock time in each FL round. This allows for more iterations to take place within the limited network connection time of vehicles.
- In FL, it is not feasible to probe the data distribution of each client. To assess the degree of data distribution differences among different clients, we divide the local data set of each vehicle client into training and testing sets with an $8:2$ ratio. In each round t, each client k receives the initial global model and begins by conducting local testing to obtain its local mean absolute error MAE_k^t. The local MAE_k^t inherently reflects the degree of inconsistency between the local client data and the global model, which can be further used to estimate the degree of bias in local updates. Here, we utilize only a portion of the training data to reduce client computational costs. After this, the vehicle client k performs local training and, based on its current contention window size, uploads its local model and testing mean absolute error MAE_k^t.

C. MADDQN Agents Training Process

In the PreM-FedIoV framework, each vehicle client k, relies on a MADDQN agent at the central server to determine its contention window size in each round. The MADDQN agent is composed of a single-layer LSTM and a simple two-layer multi-layer perceptron. It independently trains each agent, where multiple agents negotiate through sharing rewards and global states. Specifically, multiple agents communicate by uploading their local observations to the server based on the current contention window. Each MARL agent infers its action a_k^t based on the global state to allocate different access priorities to vehicle clients. The central server calculates the team reward r_t based on the global model's test MAE, the number of participating clients, and the communication wall clock time per round. This team reward is shared among all agents.

D. Design of MADDQN Agent States

In each round t, all clients request the global model, then conduct testing on their local testing sets to obtain the local MAE before proceeding with local training. After completing local training, each client k will send its local testing MAE MAE_k^t and local model to the RSU based on its current contention window CW_k^t. These two pieces of information will be included in the same data packet and sent simultaneously. The RSU will then proceed to upload the received data packets to the central server. where the local MAE of client k is updated when the data packet is successfully received by the RSU. Additionally, the central server maintains a state collection module that includes a collision list *P*, an MAE list *MAE*, and a contention window list *CW*. The collision list is used to record whether the local information from each client in each round was lost due to collisions during the upload process. The contention window list records the contention window value for each client.

When all clients have finished transmitting, the MADDQN agent in the central server collects global state information from the state collection module as part of its own state. The state of each agent in each round consists of four components. We denote the vector S_k^t to represent the state of agent k in round t,

$$S_k^t = \left\{\mathrm{ID}_k, MAE^t, P^t, \mathrm{CW}^t\right\}. \tag{6}$$

Here, $\mathrm{ID}_k = k, k \in [1, \ldots, K]$. It represents the ID of client k. $MAE^t = \{MAE_1^t, MAE_2^t, \ldots, MAE_K^t\}$ represents the MAE list in round t, recording the local testing MAE of all clients. $CW^t = \{\mathrm{CW}_1^t, \mathrm{CW}_2^t, \ldots, \mathrm{CW}_K^t\}$ represents the contention window values list in round t, recording the current contention window values of all clients. $P^t = \{P_1^t, P_2^t, \ldots, P_K^t\}$ represents the collision list in round t, recording the local information upload status of all clients under the contention window.

E. Description on Agent Actions

Let a_k^t represent the action of agent k in round t. The MADDQN agent k outputs an integer between 0 and N based on the current state S_k^t, that is, $a_k^t \in [0, N]$ where N is the maximum contention window. The agent's actions directly correspond to setting new CW values. When the joint action $a_t = \{a_1^t, a_2^t, \ldots, a_K^t\}$ is executed, for each vehicle client k adopting the new contention window a_k^t, the environment transitions from state S^t to S^{t+1}, where $S^t = \{S_1^t, S_2^t, \ldots, S_K^t\}$ represents the state of all agents in round t.

Reviewing the CSMA/CA protocol in IEEE 802.11p, the initial value of CW is $\mathrm{CW}_{\min}$, each time a station retransmits due to a transmission failure, the CW doubles. When the value of CW increases to $\mathrm{CW}_{\max}$, during subsequent retries, CW remains at $\mathrm{CW}_{\max}$. To mitigate the increase in wireless access delay caused by directly doubling the contention window, after the agent k produces an action decision, we set both the $\mathrm{CW}_{\min}$ and $\mathrm{CW}_{\max}$ contention windows to be a_k^t.

F. Design of the Reward Function

In order to optimize the performance of the FL framework in IoV which is described in (5), in each round t, we aim for MADDQN agents to generate wireless access priority decisions that result in lower global model testing MAE and achieve a shorter iteration communication wall clock time. Additionally, to reduce the total communication energy consumption, we prefer to have as few vehicle clients as possible participating

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

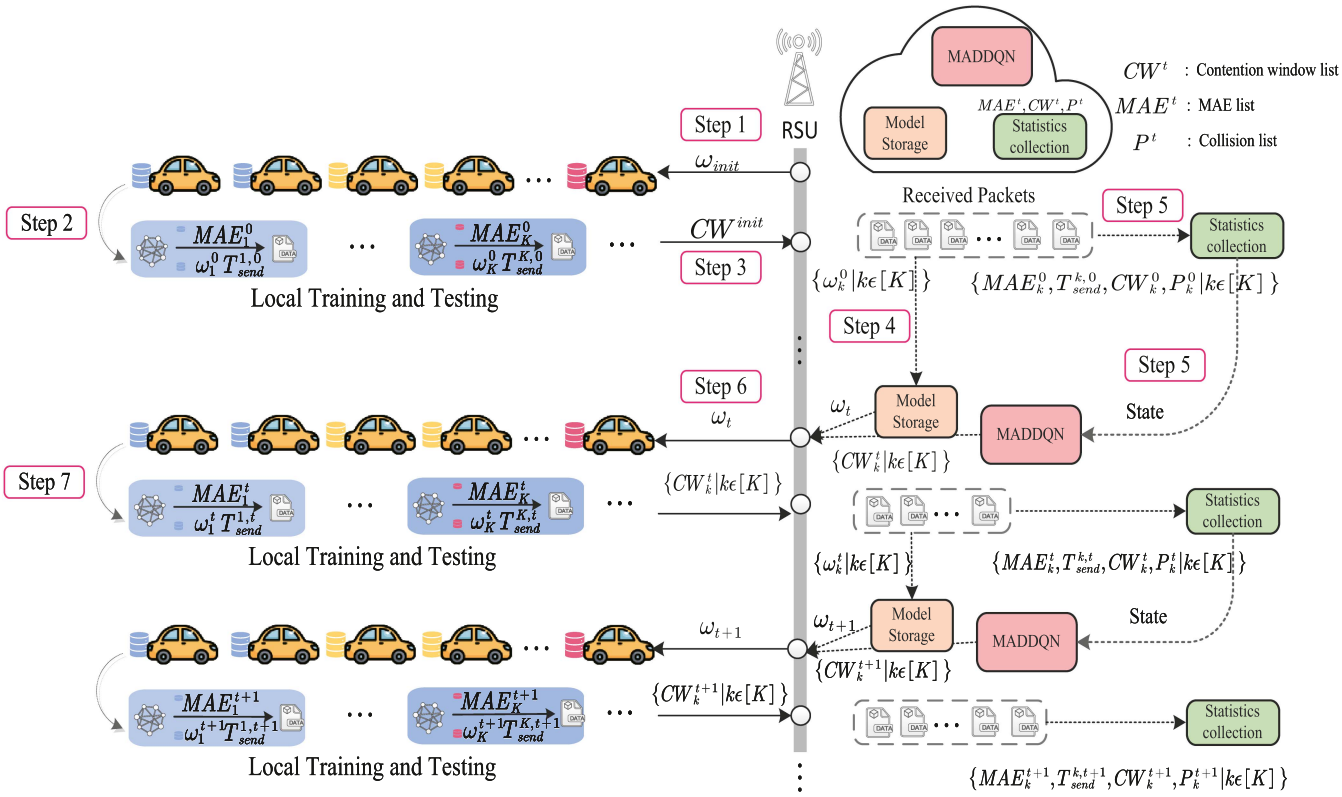


Fig. 2. Workflow of PreM-FedIoV.

in each federated aggregation. Therefore, we set the observed reward at the end of each round t as follows

$$r_t = -w_1[\Xi^{(MAE(t)-\Omega)} - 1] - w_2 \cdot T^t - w_3 \cdot C_t. \quad (7)$$

In each round t, when vehicle client k uploads a packet containing its local model and local testing MAE data, it attaches a current timestamp. When the central server receives this data packet, it records the time of reception. Furthermore, $C_t = \sum_{k=1}^{K} P^t$, represents the summation of the collision list, signifying the count of clients participating in federated aggregation in round t. $MAE(t)$ represents the testing MAE achieved by the global model on the held-out validation set after round t, where Ω serves as the target testing MAE, and Ξ functions as a constant that ensures controlled growth. All MADDQN agents share the same reward and are trained to maximize the expected cumulative discounted reward given by

$$Reward = \sum_{t=1}^{T} \gamma^{t-1} r_t, \quad (8)$$

where $\gamma \in (0, 1]$ is a factor discounting future rewards.

G. Workflow of PreM-FedIoV

Fig. 2 illustrates how our FL framework where $k \in [K]$ means $k \in [1, \ldots, K]$, PreM-FedIoV, adaptively adjusts the contention window size for each vehicle client in each round in collaboration with the MADDQN agents. The central server oversees three key modules: (1) a model storage block for storing and updating the global PreM model, (2) a MADDQN block tasked with executing trained MARL agents and determining contention window sizes for vehicle clients, and (3) a statistics collection block dedicated to gathering local information from vehicle clients.

It is worth noting that the statistics collection model comprises three lists: one for storing the local test MAE of all clients (MAE list), another for collecting collision information during transmission (collision list), and a third for storing the current contention window sizes of all clients (contention window list). The client's local test MAE in the MAE list is only updated when the FL server successfully receives it, ensuring that no extra communication overhead is introduced. The specific steps are as follows,

- *Step 1:* All vehicles request the initial global model ω_{init} from the central server, and the central server sends the initial global model to the vehicle clients through the RSU.
- *Step 2:* Each vehicle client first obtains the local test MAE MAE_k^0 on its local test dataset, then proceeds with local model training on its local training dataset to obtain the local model ω_k^0.
- *Step 3:* For round $t \in [0, T]$, every client packages its local model, local test MAE, and a current timestamp into a data packet, subsequently transmitting this local information to the central server via the RSU using the current contention window in accordance with IEEE 802.11p, the initial contention window size for all vehicle clients in the first round is set to 15.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

- *Step 4:* The statistics collection block reads the content of received data packets, sends the received local models to the model storage block, which averages the received local models and generates a new global model ω_{t+1}.
- *Step 5:* The statistics collection block records the reception time of each data packet, calculates the communication latency for each round according to Formula 3 for reward computation, observes collisions to generate the collision list P^0, and keeps track of the contention window sizes for all vehicle clients while generating the MAE list MAE^0. Subsequently, it sends P^0, CW^0, and MAE^0 to the MADDQN block, where each agent generates the contention window size for the $t+1$ communication round of each client based on their current state.
- *Step 6:* For round $t+1$, each vehicle client requests the global model ω_{t+1} and its own contention window size for communication from the central server, and proceeds to repeat steps $1-5$.

H. Complexity Analysis

The time complexity of MADDQN is primarily determined by the number of parameters in its neural network layers. Each agent consists of a single-layer LSTM and a simple two-layer multi-layer perceptron. Assume the input feature dimension is d_1, and the hidden state dimension is d_2. LSTM maintains a total of 4 sets of parameters, corresponding to the input gate, the output gate, the forget gate, and the candidate memory cell, respectively. So the time complexity of the LSTM layer is $O(4(d_1 \cdot d_2 + d_2^2))$. Assume m_1 is the number of neurons in the hidden layer of the perceptron, and the action space size is N. The time complexity of two-layer multi-layer perceptron is $O(d_2 \cdot m_1 + m_1 \cdot N)$ Therefore, the time complexity of MADDQN algorithm is $O(4d_1 \cdot d_2 + 4d_2^2 + d_2 \cdot m_1 + m_1 \cdot N)$

V. Implementation of PreM-FedIoV in NS-3

When explaining the impact of wireless channel conditions on model convergence, there is a significant gap in FL experiments. Many FL schemes are studied under assumptions of wireless communication settings, but none effectively evaluated IoV scenarios involving packet loss or degraded service quality. To address this challenge, we leverage NS3-gym as a bridge to implement FL within the NS-3 network simulator. In this section, we first provide an overview of how PreM-FedIoV is implemented in NS-3. Then, we describe the creation of the IoV network in NS-3.

A. Overview

Fig. 3 illustrates the overall architecture of PreM-FedIoV implementation in NS-3. NS-3 is an open-source project developed in C++ under the GNU GPLv2 license. The simulator classes adeptly abstract protocol stacks and comprehensive support provided by built-in classes for an array of networking protocols (like IP, TCP, UDP), communication technologies (such as Ethernet, Wi-Fi, LTE), and statistical models encompassing channels, mobility, and traffic generation. Moreover,

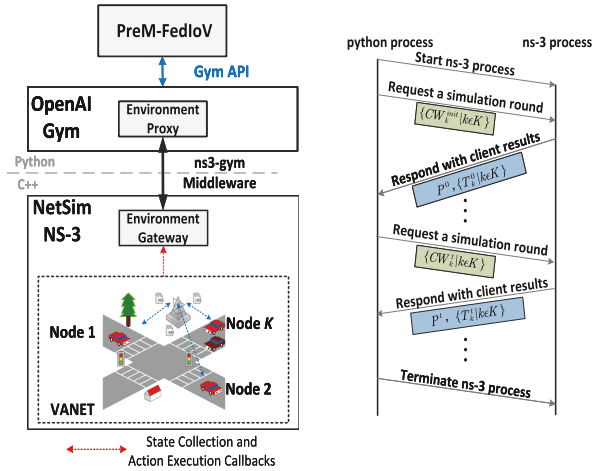


Fig. 3. Implementation of PreM-FedIoV in NS-3.

we use NS3-gym to bridge the gap between FL frameworks, RL agents developed in Python (e.g., PyTorch, TensorFlow), and NS-3 network environments. This middleware, depicted in Fig. 3, facilitates seamless interaction between MARL agents and the NS-3 environment. It consists of two components: the environment gateway, inside NS-3, collects states and translates actions into function calls. The environment proxy conveys states to agents through the Python Gym API. Specifically, to begin with, we assemble NS-3's network components to create an IoV network model using standard NS-3 functions. Subsequently, we instantiate an NS3-gym gateway within the simulation to facilitate joint operations for MADDQN and implement callback functions to collect environment states shared with MADDQN agents. We implement PreM-FedIoV using the PyTorch framework and deploy NS3-gym environment agents. MADDQN agents interact with the environment using the standard *Gym::step(action)* function.

At the start of each FL training round, PreM-FedIoV sends a network simulation request to NS-3. Within the framework, each client initially performs local training. After completing the local training, PreM-FedIoV compiles a list of contention window sizes for clients in that round. Subsequently, NS-3 proceeds to execute the network simulation for that round. Each vehicle node sends a data packet to the RSU node based on the listed contention window sizes. Following this, NS-3 reports the packet collision status P^t and communication latency T_k^t for each client in that round.

On the Python side, the global model aggregates weights based on the packet collision status. MADDQN infers the contention window size for the next round's clients based on the current state and iterates until convergence.

B. Network Model

In this work, we employ the NS-3 simulator to simulate the data transmission functionality required to represent the end-to-end FL transaction. To simulate the IoV, we instantiate $K+1$ NS-3 nodes for K simulated vehicles and a RSU. In NS-3, nodes represent mobile transceivers capable of sending and

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION CONFIGURATION

Parameters	Value
Simulator	NS3.38
MAC Protocol	IEEE 802.11p
Radio Propagation Model	Friis
Bandwidth	20MHz
Simulation Area	$200m \times 200m$
Number of Vehicles	20
Number of RSU	1
N	4
Data type	Constant Bit Rate
Transmission Power	15 dBm

receiving data packets to and from other nodes in the simulated network. Each node includes a data link layer, a physical layer, an application layer, a network layer and a mobility model.

Specifically, we configure the MAC layer parameters of nodes under the IEEE 802.11p standard by invoking the *default()* method of the *ns3::NqosWaveMacHelper* class. Furthermore, we use *ns3::YansWifiChannelHelper* and *ns3::YansWifiPhyHelper* to construct the channels and physical layers of nodes. We set the propagation loss model to the Friis propagation loss model, with a working frequency of 5.9 GHz, which is commonly used for vehicular communication under the IEEE 802.11p standard. For the network layer, we utilize the *ns3::Ipv4AddressHelper* class to assist in configuring IPv4 addresses and employ the *ns3::InternetStackHelper* class to configure IPv4 protocol stacks for each node, connecting the nodes to the simulated Internet. At the application layer, we custom-design an application whereby whenever a vehicle node completes local training, it will send a data packet to the RSU node based on the current contention window size.

In addition, to simulate the movement of vehicles within the small cellular network, we equipped K vehicle nodes with the *ns3::RandomWalk2dMobilityModel* mobility model. Each vehicle node will move uniformly at a speed of 80 kilometers per hour within a rectangular boundary of 200 meters by 200 meters. If it encounters one of the boundaries of the model, it will bounce back on the boundary with a reflection angle and velocity. The RSU node is located at the center of the rectangle and equipped with the *ns3::ConstantPositionMobilityModel* mobility model. Once the position is set, it will remain stationary throughout the simulation period without changing.

VI. DATASET AND PROCESSING

Due to the automotive industry often treating vehicle sensor data as highly confidential, there are relatively few publicly available real-world datasets for automotive data [53]. The NASA C-MAPSS dataset [54] is widely used for predicting remaining useful life and is also applied in research for PreM in the automotive field [55]. The C-MAPSS dataset comprises sensor data from multiple turbine engines, each exhibiting diverse operating and health states. Specifically, the dataset consists of four sets of degradation data, each associated with a different number of engines. Additionally, each engine within the dataset has a unique RUL value, representing the remaining operational time until failure.

On the other hand, the design of battery management systems is crucial for cost reduction, improving efficiency, and enhancing safety in electric vehicles. The NASA Randomized Battery Usage dataset [56] finds widespread applications in the field of electric vehicle health management [57], [58]. It is categorized into 7 groups based on charging/discharging profiles and the utilized temperature. Each group of batteries undergoes distinct randomized cycling processes. Every 50 cycles, a series of reference charge and discharge cycles is executed to assess the health status of the batteries. Therefore, in this paper, we experiment with these two datasets.

A. Unbalanced and Non-IID Data

For the C-MAPSS dataset, in order to construct non-IID distributed client-local datasets, we select distinct sample features for K clients, with K set to 20. Specifically, for clients 1-10 and the test dataset, the sample features consist of signals from sensor 1 to sensor 20, while for clients 11-20, the sample features include 3 operational settings and signals from sensor 1 to sensor 17. We select a look-back window of 25 and divide the time series data into continuous sequences with a shape of 25×20. Furthermore, to create the imbalanced client dataset, we divided the engine IDs 1-100 into 20 unequal portions, thereby obtaining a local dataset for each client.

For the NASA Randomized Battery Usage dataset, each row represents a unique charging, discharging, or resting step in a battery cycling experiment. Each step includes eight fields: a string description of the step, its type, sampling time, voltage, current, temperature, date, and the sampling time in seconds. We utilize the mean and variance of voltage, current, and temperature within each step as sample features and apply min-max normalization to standardize them between 0 and 1. The data is segmented into continuous sequences of 100×6 using a lookback window of 100. For training, we randomly selected 20 lithium-ion batteries, corresponding to 20 vehicle clients, and for testing, we randomly chose one lithium-ion battery. The information of the local datasets for each client is shown in Table II.

B. Data Labeling

For the C-MAPSS dataset, we utilize two degradation methods, specifically linear degradation and piecewise degradation, to determine the RUL value in the label. The linear degradation method is defined as

$$y = R_{\text{max}} - R_i, \tag{9}$$

where R_{max} corresponds to the full life cycle of each engine, and R_i corresponds to the last operating cycle of the current window period. The degradation method utilizes a threshold value to determine the onset of degradation. It assumes that the engine operates in a healthy state initially, without any degradation. However, degradation starts to occur once the RUL falls below the threshold value after a certain period of operation. We denote

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II
UNBALANCED AND NON-IID DATA AMONG CLIENTS

Client ID	1	2	3	4	5	6	7	8	9	10
Dataset Size *(C-MAPSS)*	456	603	2150	1645	1038	602	320	173	2297	824
Dataset Size *(Battery)*	608	1209	753	772	1114	1128	1133	1275	1208	1238
Sample Features*(C-MAPSS)*	$Sensor1, Sensor2, Sensor3, ..., Sensor20$									
Client ID	11	12	13	14	15	16	17	18	19	20
Dataset Size*(C-MAPSS)*	933	1241	185	594	636	241	2559	496	145	993
Dataset Size *(Battery)*	519	500	507	611	663	602	901	1397	527	1173
Sample Features*(C-MAPSS)*	$Setting1, Setting2, Setting3, Sensor1, ..., Sensor17$									

this degradation method by the following formula

$$y = \begin{cases} R_{thres}, & \text{if} \quad 0 \le R_i \le R_{thres}; \\ R_{\max} - R_i, & \text{if} \quad R_{thres} \le R_i \le R_{\max}. \end{cases} \tag{10}$$

According to [59], we set the value of R_{thres} as 130 in our experiments.

Due to the lack of practical significance for defining RUL as the remaining cycle count for lithium-ion batteries, for the NASA Randomized Battery Usage Dataset, we define the RUL value in the labels as the standardized remaining ampere-hours (Ah) that a battery can provide before reaching the EOL [57]. The standardized remaining ampere-hours is defined as

$$y = [trapz(\mathbb{C}, \mathbb{T}) - trapz(\mathbb{C}_{\{i:EOL,:\}}, \mathbb{T}_{\{n:EOL,:\}})]/nomcap, \tag{11}$$

i represents the current cycle number, $trapz$ denotes the approximate integral calculated using the trapezoidal method, $\mathbb{C}$ refers to the current matrix, $\mathbb{T}$ represents the matrix corresponding to the elapsed time for the current measurement, and $nomcap$ represents the nominal capacity of the battery, which is 2.2Ah in this dataset.

VII. DEEP LEARNING FOR PREM

CNNs excel at recognizing complex patterns by exploiting hierarchical patterns present in the data. Therefore, CNNs are widely used for predicting the remaining service life of machines and equipment. On the other hand, LSTM networks are composed of units with input gates, output gates, and forget gates. LSTM units can retain information over arbitrary time intervals and control the flow of information through gates, enabling them to capture long-term temporal dependencies, which is advantageous for sensor time series data. Recognizing the limitations of available data samples and computational resources on vehicles, we design lightweight CNN and LSTM models tailored to two different datasets.

A. Local Model for the NASA C-MAPSS Dataset

For the NASA C-MAPSS dataset, the CNN model is designed with a single-layer Conv1d, utilizing 5 output channels, a kernel size of 3, and a padding value of 1 to extract features. Following the Conv1d layer, a pooling layer with a pooling core of 2 is applied to reduce dimensionality. Additionally, a dropout layer with a dropout probability of 0.5 is employed to randomly discard features. The resulting features are then flattened into a one-dimensional vector and passed through a fully connected layer. Finally, another fully connected layer is utilized to generate the output.

The LSTM model utilizes an LSTM layer with a hidden state size of 5. This allows the model to process inputs across multiple time steps and generate both output and hidden states. The output of the last time step is selected from the LSTM output and used as the input for the fully connected layer, mapping the extracted features to the final result. The performance of these two networks was evaluated on the RUL prediction task.

The local model uses Mean Square Error (MSE) as the loss function for model training, which can be represented as

$$L(f_{\boldsymbol{w}}(x), y) = \sqrt{\frac{1}{M}\sum_{i=1}^{M}(f_{\boldsymbol{w}}(x) - y)^2}. \tag{12}$$

Here, M represents the number of local samples and y represents the RUL corresponding to each window period.

B. Local Model for the NASA Randomized Battery Usage Dataset

For the NASA Randomized Battery Usage Dataset, the CNN model consists of two Conv1d layers followed by two fully connected layers. The input data has 6 channels, which, after passing through the first convolutional layer, increases to 64 channels, and then decreases to 32 channels after the second convolutional layer. The SELU activation function is applied after each convolutional layer. Subsequently, the output of the convolutional layers is mapped to the final output dimension through two fully connected layers.

The LSTM model comprises two LSTM layers. The first LSTM layer transforms the input sequence into a feature representation with 64 hidden units, while the second LSTM layer reduces the number of hidden units to 32. The SELU activation function is applied after each LSTM layer. Afterward, the output dimensions of the LSTM layers are projected onto the final output space via two fully connected layers.

The training of the local model is based on the Smooth L1 loss function, which can be represented by

$$\text{SmoothL1}(f_{\boldsymbol{w}}(x), y) = \begin{cases} 0.5(f_{\boldsymbol{w}}(x) - y)^2, |f_{\boldsymbol{w}}(x) - y| < 1 \\ |f_{\boldsymbol{w}}(x) - y| - 0.5, \text{otherwise}. \end{cases} \tag{13}$$

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

VIII. Experiments, Results and Discussion

In this section, we provide a comprehensive analysis of our experiments on the RUL prediction tasks for the C-MPASS and Randomized Battery Usage datasets using the PreM-FedIoV framework in NS-3. As a baseline, we choose the CSMA/CA mechanism from the standard IEEE 802.11p. We first compare the FedAvg and FedAdam frameworks under different MAC mechanisms, and then within different FL frameworks, we replace different local models and labeling methods to investigate their respective performance. The experimental results show that our framework consistently achieves lower testing MAE compared to the baseline. Moreover, there is a reduction in the communication wall clock time and the number of participating devices in each round, indicating that PreM-FedIoV effectively reduces communication overhead while improving testing accuracy.

A. Experiment Setup

We set the number of vehicle clients K to 20 and use the data partitioning method described in part A of Section VI to divide the dataset into 20 clients, with each client's local dataset being non-IID and imbalanced. As discussed in Section VII, our experiments utilize either CNN or LSTM as a local model for processing sequence samples. The local model processes 25×20 sequence samples for the CMAPSS dataset, and 100×6 sequence samples for the Randomized Battery Usage dataset.

During the training process, we establish a MAE target threshold within the PreM-FedIoV framework. Specifically, we set the target test MAE Ω to 15 for the C-MAPSS dataset and 0.05 for the Randomized Battery Usage dataset. This predefined target serves as guidance for the training model, enabling it to optimize its performance and approach the desired threshold.

All experiments are conducted with the following settings: the local training epoch is set to 3, the total communication rounds are set to 250, and the batch size is set to 20. Regarding the MADDQN model on the server, it comprises 20 DDQN models. In each DDQN model, the batch size is set to 5, the learning rate is 0.1, the discount factor is set to 0.9, the memory buffer size is set to 50, and the target network is updated with a frequency of 1. We implement the standard FL framework as a baseline in the IoV, following the specific process described in Section III. According to the latest version of the 802.11p standard draft [60], we set the minimum contention window for all vehicle stations to 15 and the maximum contention window to 1023.

To evaluate the performance of the PreM-FedIoV framework for RUL prediction, we utilize MAE, Root Mean Square Error (RMSE), and MSE as metrics [61], [62], [63], [64]. These three metrics are calculated using the following equations

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{14}$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{15}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{16}$$

Additionally, we employ the number of clients participating in each round and communication latency as evaluation metrics for investigating communication costs.

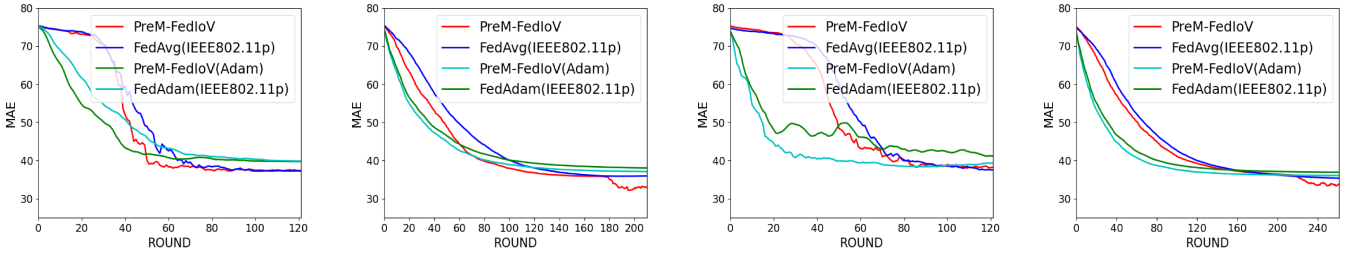
The performance of LSTM and CNN models under two different labeling methods and two different MAC mechanisms for the CMPASS dataset is illustrated in Fig. 4. Fig. 5 displays the performance of LSTM and CNN models under two different MAC mechanisms for the Randomized Battery Usage dataset. Additionally, Fig. 6 presents the final prediction results of RUL by CNN and LSTM within the PreM-FedIoV framework. Fig. 7 shows the communication latency per round under various MAC mechanisms, while Fig. 8 displays the number of clients per round in the same context. Fig. 9 illustrates the performance of different FL frameworks in the presence of stragglers caused by vehicles moving beyond the communication range. We now delve into the results, considering two key aspects: the effectiveness of RUL prediction and the associated communication cost.

B. Effectiveness of RUL Prediction Under Different MAC Mechanisms

We replace different local models for the C-MAPSS dataset and the Randomized Battery Usage dataset to validate the performance of the PreM-FedIoV framework compared to the FedAvg and FedAdam frameworks based on the IEEE802.11p protocol. Specifically, in the FedAvg and FedAdam frameworks based on IEEE802.11p, vehicle clients apply SGD and Adam as local optimizers, respectively. Then vehicle clients determine their local contention window according to the CSMA/CA MAC mechanism and upload local models to the server. To compare with the FedAvg and FedAdam frameworks, we set the local optimizer in PreM-FedIoV to SGD and Adam, respectively, to validate the performance superiority of the FL framework based on its MAC mechanism over the IEEE802.11p protocol-based frameworks. For the C-MAPSS dataset, we adopt two different labeling methods, with the local learning rates configured as $3.5e-7$ for CNN models and $7e-6$ for LSTM models. For the Randomized Battery Usage dataset, the local learning rate configuration for CNN models is $6e-8$, and for LSTM models, it is $2e-6$.

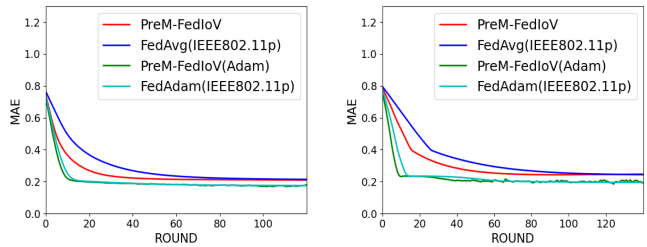
The performance figures achievable on the C-MAPSS dataset are showna in Table IV and Fig. 4, while the performance figures achievable on the Randomized Battery Usage dataset is shown in Table III and Fig. 5. For simplicity, we use FedIoV to represent the PreM-FedIoV framework, FedIoV (Adam) to denote the PreM-FedIoV framework where the local optimizer is replaced with the Adam optimizer, and Linear and Piecewise to represent the two labeling methods.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.



(a) CNN - Linear degradation (b) LSTM - Linear degradation (c) CNN - Piecewise degradation (d) LSTM - Piecewise degradation

Fig. 4. MAE vs Rounds across different MAC mechanisms, labeling methods, and network architectures for the C-MAPSS dataset.



(a) CNN (b) LSTM

Fig. 5. MAE vs Rounds across different MAC mechanisms and network architectures for the Randomized Battery Usage dataset.

TABLE III
COMPARING METRICS OF LSTM AND CNN MODELS UNDER DIFFERENT MAC MECHANISMS ON THE RANDOMIZED BATTERY USAGE DATASET

FedAvg			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	0.2433	0.0855	0.2924
CNN	0.2102	0.0708	0.2662
FedAdam			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	0.1934	0.0607	0.2465
CNN	0.1654	0.0447	0.2113
FedIoV			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	0.2433	0.0854	0.2922
CNN	0.2071	0.0685	0.2617
FedIoV(Adam)			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	0.1846	0.0569	02385
CNN	0.1733	0.0499	0.2235

We can observe that, for the CMAPSS dataset, when the local optimizer is SGD and the labeling method is Linear, if the local model is LSTM, the PreM-FedIoV framework compared to FedAvg (IEEE 802.11p) has improved the MAE, MSE, and RMSE metrics by 10.22%, 22.93%, and 12.22% respectively. Under the Piecewise labeling method, when the local model is CNN, the three metrics have increased by 1.2%, 1.9%, and 0.9% respectively. In other cases, the performance is comparable to FedAvg. When the local optimizer is Adam, if the local model is LSTM and the labeling method is Piecewise, the performance of these three metrics has increased by 4.55%, 3.5%, and 1.75% respectively. For the Randomized Battery Usage dataset, when the local optimizer is SGD and the local model is CNN, the three metrics have increased by 1.47%, 3.24%, and 1.69% respectively. When the local optimizer is Adam and the local model is LSTM, the three metrics have increased by 2.42%, 1.78%, and 0.87% respectively. Furthermore, in all scenarios of the two datasets, the convergence rate of the PreM-FedIoV framework is consistently lower than the baseline. This implies that in vehicular networking scenarios, PreM-IoV can achieve comparable global model performance with lower communication costs.

This can be attributed to the following facts: the local dataset of each vehicle client is imbalanced and non-IID. Under the standard IEEE 802.11p settings, all client media access priorities are the same, collisions occur randomly, and it's easy to get stuck in local optima. However, in the proposed PreM-FedIoV framework, different clients have varying priorities for media access, depending on the degree of bias on the local updates. Clients with a lower degree of bias enjoy higher priority, while those with a higher degree of bias are assigned lower priority, resulting in the inability of these local models to be uploaded successfully for aggregation. To some extent, this is akin to aggregating from clients with a lower degree of bias in each round, which helps accelerate convergence and achieve better performance.

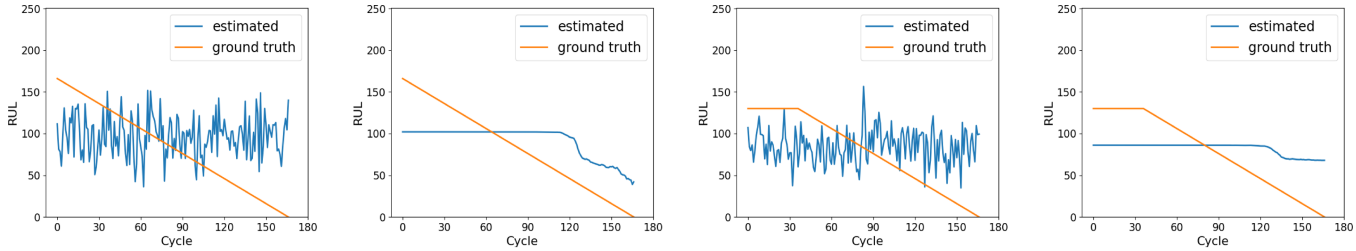
C. Effectiveness of RUL Prediction With Different Local Optimizers

From Figs. 4, 5, and Tables III and IV, we investigate the performance of two different FL frameworks, FedAvg and FedAdam, under different MAC mechanisms for C-MAPSS dataset and Randomized Battery Usage dataset.

We observe that for the Randomized Battery Usage dataset, when the local optimizer is Adam, compared to SGD, the MAE, MSE, and RMSE metrics respectively increase by 24.12%, 33.37%, and 18.37% when the local model is LSTM. When the local model is CNN, these three metrics respectively increase by 16.32%, 27.15%, and 14.59%.

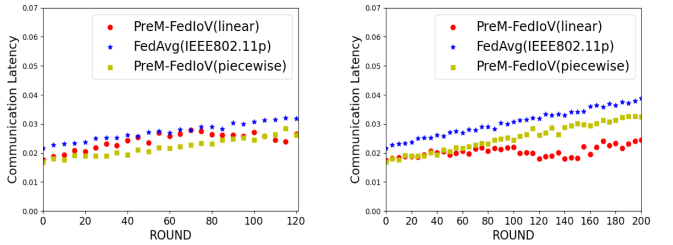
However, for the C-MAPSS dataset, although Adam as the local optimizer leads to faster convergence, its performance metrics still fall behind SGD. Additionally, regardless of whether

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

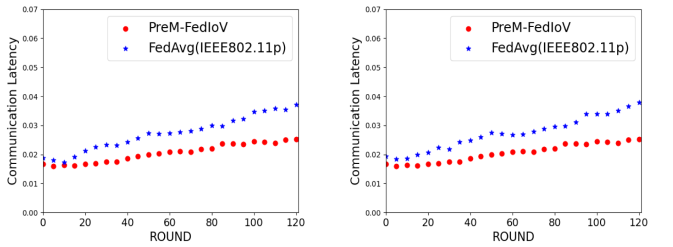


(a) CNN-Linear degradation (b) LSTM-Linear degradation (c) CNN-Piecewise degradation (d) LSTM-Piecewise degradation

Fig. 6. Predicted degradation trajectory of engine 1 under different network architectures.

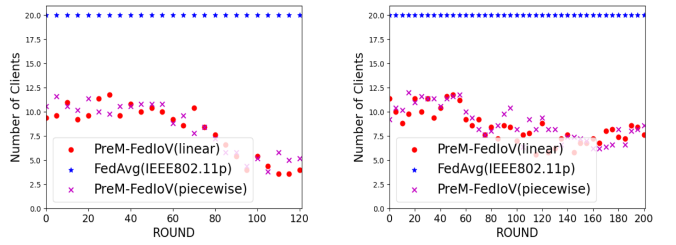


(a) CNN as the local model for C-MAPSS dataset (b) LSTM as the local model for C-MAPSS dataset

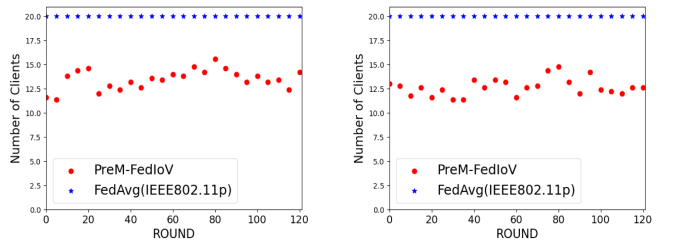


(c) CNN as the local model for Battery dataset (d) LSTM as the local model for Battery dataset

Fig. 7. Communication delay per round under different MAC mechanisms.



(a) CNN as the local model for C-MAPSS dataset (b) LSTM as the local model for C-MAPSS dataset



(c) CNN as the local model for Battery dataset (d) LSTM as the local model for Battery dataset

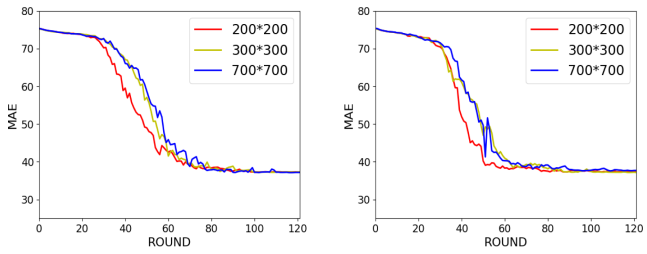
Fig. 8. Number of clients per round under different MAC mechanisms.

the local optimizer is SGD or Adam, PreM-FedIoV consistently achieves convergence at a faster rate.

D. Effectiveness of RUL Prediction With Different Local Models

For the C-MAPSS dataset, we utilize the degradation curve of Engine 1 as the fitting target. A well-trained global model is employed to predict the Remaining Useful Life (RUL) under different operating conditions, as illustrated in Fig. 6. In this figure, the orange curve represents the actual RUL under real conditions, while the green curve represents the RUL inferred by CNN and LSTM networks. Observations reveal that compared to CNN, the curve generated by LSTM is notably smoother and exhibits higher fitting accuracy.

Furthermore, the effectiveness of RUL prediction by different local models can be observed in Figs. 4 and 5, as well as Tables III and IV. For the C-MAPSS dataset, when the local optimizer is SGD and the labeling method is linear, using LSTM as the local model instead of CNN improves the MAE, MSE, and RMSE metrics by 13.56%, 28.84%, and 15.65% respectively.



(a) FedAvg(IEEE 802.11p) (b) PreM-FedIoV

Fig. 9. MAE vs Rounds for different motion range.

When the labeling method is Piecewise, these metrics improve by 5.75%, 7.33%, and 3.72% respectively. However, for the random battery usage dataset, when the local optimizer is SGD and the labeling method is linear, using CNN as the local model instead of LSTM increases the MAE, MSE, and RMSE metrics by 14.87%, 19.78%, and 10.43% respectively. Additionally, for both datasets, CNN converges faster than LSTM.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARING METRICS OF LSTM AND CNN MODELS UNDER DIFFERENT MAC MECHANISMS WITH TWO LABELING METHODS ON THE C-MAPSS DATASET

FedAvg Linear			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	35.90	1809.63	42.54
CNN	37.25	1929.32	43.92
FedAvg Piecewise			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	35.37	1690.82	41.12
CNN	38.01	1790.75	42.31
FedAdam Linear			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	37.10	1732.74	41.62
CNN	39.73	2249.92	47.43
FedAdam Piecewise			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	37.07	1685.82	41.05
CNN	36.05	1674.97	40.92
FedIoV Linear			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	32.23	1394.74	37.34
CNN	37.29	1960.22	44.27
FedIoV Piecewise			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	35.38	1626.65	40.33
CNN	37.54	1755.35	41.89
FedIoV(Adam) Linear			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	38.04	1794.33	42.35
CNN	39.78	2367.55	48.65
FedIoV(Adam) Piecewise			
Model	$\mathcal{MAE}$	$\mathcal{MSE}$	$\mathcal{RMSE}$
LSTM	36.17	1655.68	40.69
CNN	36.65	1869.38	43.23

E. Effectiveness of Communication Delay Under Different MAC Mechanisms

To study the communication latency in each round across different frameworks, we keep the local optimizer for clients fixed as SGD. We replace different labeling methods and local models in the framework for the CMAPSS dataset, and for the Randomized Battery Usage dataset, we substitute different local models. Additionally, We apply a low-pass filter with an averaging time window of 5 data points. Fig. 7 shows the communication latency per round for both MAC mechanisms when CNN and LSTM are used as local models. We can observe that the proposed PreM-FedIoV framework achieves lower communication latency per round. For the CMAPSS dataset, when the local model is LSTM and the labeling method is Piecewise, the communication latency per round averaged by FedAvg based on IEEE 802.11p is 0.0203 s, while for the FedIoV framework, the latency is 0.020 s. Compared to the setting under IEEE 802.11p, our framework can reduce the average communication latency per round by 32.87%. For the Battery Random Usage dataset, when the local model is LSTM, the average communication latency per round by FedAvg based on IEEE 802.11p is 0.0268 s, while for the FedIoV framework, the latency is 0.0203 s. Compared to the setting under IEEE 802.11p, our framework can reduce the average communication latency per round by 24.44%. This is because, under the IEEE 802.11p setting, all vehicle nodes have the same contention window size. When a collision occurs, the contention window size doubles, and retransmissions are required until reaching the maximum retry limit.

However, in the PreM-FedIoV framework, smaller contention window sizes are allocated to vehicle nodes exhibiting a higher degree of bias, whereas those with a lower degree of bias receive larger contention window sizes. This ensures that vehicles with a lower degree of bias have a higher likelihood of successful transmission compared to fixed initial contention windows in IEEE 802.11p, thus avoiding retransmissions. On the other hand, vehicles with a higher degree of bias are assigned smaller contention window sizes, resulting in reduced transmission time.

F. Number of Participating Clients Under Different MAC Mechanisms

We study the number of participating clients per round under different MAC mechanisms, and this number has a direct impact on communication overhead. Fig. 8 presents the specific results, and in this experiment, we also apply a low-pass filter with an average time window of 5 data points. We observe that under the IEEE 802.11p setting, due to the contention window doubling after each collision, the local models of all clients participate in aggregation in every round. In contrast, the PreM-FedIoV framework introduces controlled collisions, resulting in the participation of only a subset of clients in each round, effectively reducing communication overhead. Additionally, the number of clients participating in each round is variable. This is because we dynamically allocate different sizes of contention window size for different clients in each round through MADDQN, thus the number of local models participating in the aggregation is dynamic. For the C-MPASS dataset, when the labeling method is Linear and the local model is LSTM, on average, only 8.48 clients participate in FL communication per round, which represents a 57.6% improvement compared to the baseline. For the Random Battery Usage dataset, when the local model is LSTM, on average, only 12.68 clients participate in FL communication per round, which represents a 36.58% improvement compared to the baseline.

It is worth noting that as the number of iterations increases, the participation of clients decreases under both labeling methods and local model settings. This trend occurs due to training convergence, as the PreM-FedIoV framework can adaptively adjust the number of participating clients during the training process.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

G. The Impact of Vehicle Mobility on Federated Learning Performance

We utilize the *ns3::RandomWalk2dMobilityModel* mobility model to simulate vehicle mobility, with the transmission power of vehicle nodes set to 15 dBm. We adjust the vehicle motion range to 300x300 and 700x700 while maintaining a fixed random seed and initial vehicle positions. In this setup, the initial travel paths of the vehicles remain consistent until they rebound upon reaching the boundaries. Furthermore, we maintain a fixed labeling method of linear degradation and employ CNN as the local model.

Fig. 9 illustrates the MAE performance of the FedAvg(IEEE 802.11p) and PreM-FedIoV frameworks concerning the mobility of vehicles. We observe that, compared to the 200x200 motion range, the other two settings require more communication rounds to achieve convergence. This is because some vehicles are positioned beyond the current wireless communication range from the RSU, resulting in the local models failing to upload successfully. However, under the settings of 200x200, 300x300, and 700x700, the PreM-FedIoV framework converges faster than FedAvg(IEEE 802.11p). This implies that within a limited network connection time, the proposed PreM-FedIoV can adapt better to mobile scenarios.

IX. Conclusion

In this paper, we presented the design and implementation of the PreM-FedIoV framework, an experience-driven FL framework primarily applied in IoV. Traditional BEB mechanism would lead to substantial communication overhead and reduced prediction accuracy in the context of federated learning. Hence, we extended the single-agent DQN to MADQN. In each round, agents can adaptively adjust the contention window size for each vehicle to minimize communication rounds during FL training. Additionally, we recognized the impact of real network behavior on FL performance. To address this, we used NS-3 to create a realistic IoV simulation. We also built a bridge between the IoV simulation and the PyTorch-based FL framework using NS3-gym. To validate the impact of network behavior on the FL framework when deployed in IoV, we selected FedAvg as the baseline. Extensive comparisons showed that utilizing PreM-FedIoV for FL training effectively increased test accuracy while reducing the number of communication devices and communication latency.

References

[1] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, L.-N. Tran, S. Gong, and E. Dutkiewicz, "Dynamic federated learning-based economic framework for Internet-of-Vehicles," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2100–2115, Apr. 2023.

[2] R. Chaudhary and N. Kumar, "SecGreen: Secrecy ensured power optimization scheme for software-defined connected IoV," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2370–2386, Apr. 2023.

[3] Z. Mahmood, "Connected vehicles in the IoV: Concepts, technologies and architectures," in *Connected Vehicles in the Internet of Things: Concepts, Technologies and Frameworks for the IoV*. Berlin, Germany: Springer, 2020, pp. 3–18.

[4] Q. Kong, R. Lu, F. Yin, and S. Cui, "Privacy-preserving continuous data collection for predictive maintenance in vehicular fog-cloud," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5060–5070, Aug. 2021.

[5] S. Wang, Y. Fan, S. Jin, P. Takyi-Aninakwa, and C. Fernandez, "Improved anti-noise adaptive long short-term memory neural network modeling for the robust remaining useful life prediction of lithium-ion batteries," *Rel. Eng. Syst. Saf.*, vol. 230, 2023, Art. no. 108920.

[6] D. M. Manias and A. Shami, "Making a case for federated learning in the Internet of Vehicles and intelligent transportation systems," *IEEE Netw.*, vol. 35, no. 3, pp. 88–94, May/Jun. 2021.

[7] X. Zhang, Z. Chang, T. Hu, W. Chen, X. Zhang, and G. Min, "Vehicle selection and resource allocation for federated learning-assisted vehicular network," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 3817–3829, May 2024.

[8] D. Qiao, M. Li, S. Guo, J. Zhao, and B. Xiao, "Resources-efficient adaptive federated learning for digital twin-enabled IIoT," *IEEE Trans. Netw. Sci. Eng.*, pp. 1–13, 2024.

[9] L. Yang, C.-K. Tham, and S. Guo, "FedPreM: A novel federated reinforcement learning framework for predictive maintenance," in *Proc. IEEE Glob. Commun. Conf.*, 2023, pp. 7556–7561.

[10] M. Mohri, G. Sivek, and A. T. Suresh, "Agnostic federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4615–4625.

[11] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," 2019, *arXiv: 1907.02189*.

[12] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, 2020.

[13] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, and J. Liu, "Adaptive control of local updating and model compression for efficient federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5675–5689, Oct. 2023.

[14] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "A novel framework for the analysis and design of heterogeneous federated learning," *IEEE Trans. Signal Process.*, vol. 69, pp. 5234–5249, 2021.

[15] M. Khatua and S. Misra, "D2D: Delay-aware distributed dynamic adaptation of contention window in wireless networks," *IEEE Trans. Mobile Comput.*, vol. 15, no. 2, pp. 322–335, Feb. 2016.

[16] W. Wydmański and S. Szott, "Contention window optimization in ieee 802.11 ax networks with deep reinforcement learning," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2021, pp. 1–6.

[17] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[18] S. Reddi et al., "Adaptive federated optimization," 2020, *arXiv: 2003.00295*.

[19] A. Bazzi, A. Zanella, and B. M. Masini, "An OFDMA-based MAC protocol for next-generation VANETs," *IEEE Trans. Veh. Technol.*, vol. 64, no. 9, pp. 4088–4100, Sep. 2014.

[20] O. Serradilla, E. Zugasti, J. Rodriguez, and U. Zurutuza, "Deep learning models for predictive maintenance: A survey, comparison, challenges and prospects," *Appl. Intell.*, vol. 52, no. 10, pp. 10934–10964, 2022.

[21] D. Pagano, "A predictive maintenance model using long short-term memory neural networks and Bayesian inference," *Decis. Analytics J.*, vol. 6, 2023, Art. no. 100174.

[22] B. Ghasemkhani, O. Aktas, and D. Birant, "Balanced K-Star: An explainable machine learning method for Internet-of-Things-enabled predictive maintenance in manufacturing," *Machines*, vol. 11, no. 3, 2023, Art. no. 322.

[23] M. D. Dangut, I. K. Jennions, S. King, and Z. Skaf, "A rare failure detection model for aircraft predictive maintenance using a deep hybrid learning approach," *Neural Comput. Appl.*, vol. 35, no. 4, pp. 2991–3009, 2023.

[24] A. Hosseinzadeh, F. F. Chen, M. Shahin, and H. Bouzary, "A predictive maintenance approach in manufacturing systems via AI-based early failure detection," *Manuf. Lett.*, vol. 35, pp. 1179–1186, 2023.

[25] L. Wang, Y. Chen, X. Zhao, and J. Xiang, "Predictive maintenance scheduling for aircraft engines based on remaining useful life prediction," *IEEE Internet Things J.*, p. 1, 2024.

[26] A. Wahid, M. Yahya, J. G. Breslin, and M. A. Intizar, "Self-attention transformer-based architecture for remaining useful life estimation of complex machines," *Procedia Comput. Sci.*, vol. 217, pp. 456–464, 2023.

[27] Z. Fan, W. Li, and K.-C. Chang, "A bidirectional long short-term memory autoencoder transformer for remaining useful life estimation," *Mathematics*, vol. 11, no. 24, 2023, Art. no. 4972.

[28] Y. Liu, J. Chen, T. Wang, A. Li, and T. Pan, "A variational transformer for predicting turbopump bearing condition under diverse degradation processes," *Rel. Eng. Syst. Saf.*, vol. 232, 2023, Art. no. 109074.

[29] R. Chhabra, S. Singh, and V. Khullar, "Privacy enabled driver behavior analysis in heterogeneous IoV using federated learning," *Eng. Appl. Artif. Intell.*, vol. 120, 2023, Art. no. 105881.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.

[30] Z. Lian, Q. Zeng, W. Wang, D. Xu, W. Meng, and C. Su, "Traffic sign recognition using optimized federated learning in Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6722–6729, Feb. 2024.
[31] K. Yang et al., "FLM-ICR: A federated learning model for classification of Internet of Vehicle terminals using connection records," *J. Cloud Comput.*, vol. 13, no. 1, pp. 1–17, 2024.
[32] S. Zeng, B. Mi, and D. Huang, "Emergency vehicle identification for Internet of Vehicles based on federated learning and homomorphic encryption," in *Proc. IEEE 12th Data Driven Control Learn. Syst. Conf.*, 2023, pp. 208–213.
[33] X. Hu, R. Li, Y. Ning, K. Ota, and L. Wang, "A data sharing scheme based on federated learning in IoV," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 11644–11656, Sep. 2023.
[34] J. Zhang, C. Wang, and S. Li, "Energy efficient federated learning in Internet of Vehicles: A game theoretic scheme," *Trans. Emerg. Telecommun. Technol.*, vol. 34, no. 5, 2023, Art. no. e4761.
[35] S. Zafar, S. Jangsher, and A. Zafar, "Federated learning for resource allocation in vehicular edge computing-enabled moving small cell networks," *Veh. Commun.*, vol. 45, 2024, Art. no. 100695.
[36] J.-H. Kwon, D. Kim, and E.-J. Kim, "Reinforcement learning-based contention window adjustment for wireless body area networks," in *Proc. IEEE 4th Int. Conf. Big Data Analytics Practices*, 2023, pp. 1–4.
[37] C.-H. Ke and L. Astuti, *Applying Multi-Agent Deep Reinforcement Learning for Contention Window Optimization to Enhance Wireless Network Performance*. Tamworth, England: ICT Express, 2022.
[38] G. Jin, S. Lu, and S. Hu, "Optimization of throughput and fairness performance in multi-rate WLAN network," in *Proc. 4th Int. Conf. Inf. Sci. Parallel Distrib. Syst.*, 2023, pp. 505–508.
[39] N. Subash and B. Nithya, "Dynamic adaptation of contention window boundaries using deep Q networks in UAV SWARMs," *Int. J. Comput. Appl.*, vol. 46, no. 3, pp. 167–174, 2024.
[40] C.-H. Ke and L. Astuti, "Applying deep reinforcement learning to improve throughput and reduce collision rate in ieee 802. 11 networks," *KSII Trans. Internet Inf. Syst.*, vol. 16, no. 1, pp. 334–349, 2022.
[41] A. Muhammad, M. Saqib, and W.-C. Song, "Sensor virtualization and data orchestration in Internet of Vehicles (IoV)," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage.*, 2021, pp. 998–1003.
[42] J. Posner, L. Tseng, M. Aloqaily, and Y. Jararweh, "Federated learning in vehicular networks: Opportunities and solutions," *IEEE Netw.*, vol. 35, no. 2, pp. 152–159, Mar./Apr. 2021.
[43] Y. Zeng et al., "Mobility-aware proactive flow setup in software-defined mobile edge networks," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1549–1563, Mar. 2023.
[44] M. F. Pervej et al., "Mobility, communication and computation aware federated learning for Internet of Vehicles," in *Proc. IEEE Intell. Veh. Symp.*, 2022, pp. 750–757.
[45] L. Miao, K. Djouani, B. J. van Wyk, and Y. Hamam, "Evaluation and enhancement of ieee 802.11 P standard: A survey," *Mobile Comput.*, vol. 1, no. 1, pp. 15–30, 2012.
[46] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2021–2031.
[47] Y. Li, Y. Guo, M. Alazab, S. Chen, C. Shen, and K. Yu, "Joint optimal quantization and aggregation of federated learning scheme in VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 19852–19863, Oct. 2022.
[48] A. Pressas, Z. Sheng, F. Ali, D. Tian, and M. Nekovee, "Contention-based learning MAC protocol for broadcast vehicle-to-vehicle communication," in *Proc. IEEE Veh. Netw. Conf.*, 2017, pp. 263–270.
[49] M. Han, S. Khairy, L. X. Cai, Y. Cheng, and R. Zhang, "Reinforcement learning for efficient and fair coexistence between LTE-LAA and Wi-Fi," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8764–8776, Aug. 2020.
[50] S. Szott et al., "Wi-Fi meets ML: A survey on improving ieee 802.11 performance with machine learning," *IEEE Commun. Surv. Tut.*, vol. 24, no. 3, pp. 1843–1893, Third Quarter, 2022.
[51] Z. Guo, Z. Chen, P. Liu, J. Luo, X. Yang, and X. Sun, "Multi-agent reinforcement learning-based distributed channel access for next generation wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 5, pp. 1587–1599, May 2022.
[52] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 1698–1707.
[53] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Rel. Eng. Syst. Saf.*, vol. 215, 2021, Art. no. 107864.
[54] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics Health Manage.*, 2008, pp. 1–9.
[55] A. Giannoulidis and A. Gounaris, "A context-aware unsupervised predictive maintenance solution for fleet management," *J. Intell. Inf. Syst.*, vol. 60, no. 2, pp. 521–547, 2023.
[56] B. Bole, C. Kulkarni, and M. Daigle, "Randomized battery usage data set," *NASA AMES Prognostics Data Repository*, vol. 70, 2014, Art. no. 101741.
[57] A. Aliberti et al., "Comparative analysis of neural networks techniques for lithium-ion battery SOH estimation," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf.*, 2022, pp. 1355–1361.
[58] K. Fricke, R. Nascimento, M. Corbetta, C. Kulkarni, and F. Viana, "An accelerated life testing dataset for lithium-ion batteries with constant and variable loading conditions," *Int. J. Prognostics Health Manage.*, vol. 14, no. 2, 2023.
[59] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *Proc. IEEE Int. Conf. Prognostics Health Manage.*, 2008, pp. 1–6.
[60] I. C. S. L. M. S. Committee et al., "Wireless LAN medium access control (MAC) and physical layer (PHY) specifications," *IEEE Std. 802. 11-1997*, 1997.
[61] S. Gu, Q. Xi, J. Wang, P. Qiu, and M. Li, "TCN-GAWO: Genetic algorithm enhanced weight optimization for temporal convolutional network," *J. Mech. Des.*, vol. 146, pp. 1–31, 2024.
[62] Z. Du, L. Zuo, J. Li, Y. Liu, and H. T. Shen, "Data-driven estimation of remaining useful lifetime and state of charge for lithium-ion battery," *IEEE Trans. Transport. Electrific.*, vol. 8, no. 1, pp. 356–367, Mar. 2022.
[63] U. Thakkar and H. Chaoui, "Prognostic and health management of an aircraft turbofan engine using machine learning," in *Proc. IEEE Veh. Power Propulsion Conf.*, 2023, pp. 1–6.
[64] A. Botchkarev, "Performance metrics (error measures) in machine learning regression, forecasting and prognostics: Properties and typology," 2018, *arXiv: 1809.03006*.



Lu Yang received the MS degree in electronics and communication engineering from Southwest University, China, in 2021. She is currently working toward the PhD degree in computer science and technology from the College of Computer Science, Chongqing University, Chongqing. Her current research interests include intelligent edge computing, reinforcement learning, federated learning, and the Internet of Vehicles.



Songtao Guo (Senior Member, IEEE) received the BS, MS, and PhD degrees in computer software and theory from Chongqing University, Chongqing, China, in 1999, 2003 and 2008, respectively. He was a professor from 2011 to 2012 with Chongqing University and a professor from 2013 to 2018, Southwest University. At present, he is a full professor with Chongqing University. His research interests include mobile edge computing, federated learning, wireless sensor networks, and wireless ad hoc networks. He has received many research grants as a Principal Investigator from the National Science Foundation of China. He is a senior member ACM.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.



Chen-Khong Tham (Senior Member, IEEE) received the PhD and MA degrees in electrical and information science engineering from the University of Cambridge, Cambridge, U.K. He is an associate professor with the Department of ECE of the National University of Singapore. His research interests include sensor network and machine learning architectures, and real-time sensor data analytics. From 2007 to 2010, he was was the Principal Scientist and the Department Head of the Networking Protocols Department, and a Programme Manager of the Services programme. He is in the Editorial Board of the International Journal of Network Management.



Mingyan Li received the BE degree in telecommunication engineering from Jilin University, Changchun, China, in 2015 and the PhD degree from Department of Electronic Engineering, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2021. She joined the College of Computer Science, Chongqing University, in 2021 and now works as a postdoctoral research associate. Her current research interests include industrial wireless networks, software-defined networking, and time-sensitive networks for industrial internet.



Guiyan Liu received the PhD degree in computational intelligence and information processing from Southwest University of China, in 2020. She is currently an assistant researcher with the Chongqing University of China. Her research interests include data center networks, software defined networking, network function virtualization, and 5G.



Pengzhan Zhou (Member, IEEE) received the BS degree in both applied physics and applied mathematics from Shanghai Jiaotong University, Shanghai, China and the PhD degree in electrical engineering from Stony Brook University, NY, USA. He is currently a professor in the College of Computer Science with Chongqing University, Chongqing, China. His research interests include Internet of Things, federated learning, reinforcement learning, and incentivizing mechanism.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on September 28,2025 at 01:42:31 UTC from IEEE Xplore. Restrictions apply.